

V2.1: HOW THE LIVE SERVICE WORKS

Serv Audio-to-Orders: the whole process

From headset audio in a drive-thru lane to a signed order on Serv's server, a few seconds after the car pulls forward. Every step of the live path, how the sandbox differs from a real HME install, what every part of the code does, the exact webhook, which outcomes are sent, how duplicates are prevented, and how to receive it all on AWS Lambda.

27 / 28

fixtures pass with real Deepgram + Gemini through the live path (74 of 74 conversations found)

~3 s

from the crew's "pull forward" to the order being sent (median; 45 s when no closing cue is heard)

4 of 4

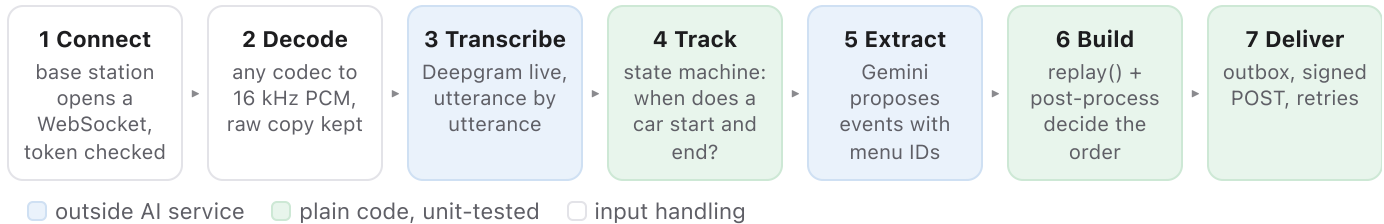
outcomes sent: completed, cancelled, abandoned, undetermined

- 1 The process on one page
- 2 One car, start to finish
- 3 Sandbox vs the real setup
- 4 What part of the code does what
- 5 The webhook, exactly
- 6 Outcomes, including abandoned
- 7 How duplicates are prevented
- 8 Receiving the webhook on AWS Lambda
- 9 Testing it yourself
- 10 Gaps and next steps
- A Commands and glossary

Prepared October 6, 2026 from the `v2.1` branch of `serv-audio-to-orders` (commit `deabaa0`, after PR #32). Companion to the October 4 architecture document, which covers v1's batch pipeline and the reasoning behind "the LLM proposes, code decides". Facts here come from the code, the README and the test suite; one behavior (section 6) was checked with a throwaway test on this date.

1 The process on one page

v1 took a finished MP3 and turned it into orders. v2 and v2.1 turn the same logic into a **live service**: an HME base station streams each lane's audio over a WebSocket while cars are ordering, and each finished conversation becomes a webhook within seconds. Files still work: they are replayed into the same lane as if they were live, so there is one path, not two.



The seven steps in plain words

1. **Connect.** The base station opens `/hme/v1/stream` on our feed service with a per-store bearer token, and names its lane and audio format (codec, sample rate, channels). Bad tokens, unknown lanes and bad formats are refused before the socket opens.
2. **Decode.** Binary messages are audio; text messages are JSON events such as `vehicle_arrived`. Every message is saved raw first (so any session can be replayed byte for byte later), then audio is decoded to 16 kHz PCM and timed by our receive clock.
3. **Transcribe.** Each lane has its own Deepgram live connection (Nova-3). Only final results become utterances; interim words go to the screen. Speaker roles come from separate channels if HME provides them, otherwise from diarization and wording.
4. **Track.** A per-lane state machine (`IDLE` → `ACTIVE` → `CLOSING` → `FINALIZED`) decides where one car's conversation starts and ends, using spoken cues ("welcome", "pull forward"), vehicle events and timers. It can reopen an order for 20 s if the customer adds something late.
5. **Extract.** One Gemini call per finished conversation. The model returns *events* (`ADD` , `REPLACE` , `CHANGE_QTY` , `CANCEL_ORDER` , `READBACK` ...) that point at catalog IDs. It never writes the order itself.
6. **Build.** The pure function `replay(events)` applies the events to produce the order. Post-processing sorts every mention into *items*, *needs review* or *not ordered*, finds combo savings, sets the outcome from evidence, and decides whether a person should check it.
7. **Deliver.** The payload is written to an SQLite outbox and POSTed at once to `WEBHOOK_URL` , signed per the Standard Webhooks spec. Failures retry for about a day before going to a dead letter queue you can resend from.

THE CORE RULE

The LLM proposes, code decides. Gemini can be wrong about what it heard, but it cannot invent a menu item, a price or a total: those come from the catalog and from code that is unit-tested. Corrections, cancellations and readbacks are deterministic.

Where it runs

FEED SERVICE: PNPM FEED SERVE

The live part. Listens on `127.0.0.1:8787`, runs one lane per store and lane, talks to Deepgram and Gemini, writes orders, and delivers webhooks. Everything it keeps goes under `.data/`.

WEB APP: PNPM DEV

Next.js on `localhost:3000`. Shows runs, orders, Test Lab, the eval, and the mock webhook receiver. Reads the same SQLite database the feed service writes. Also runs file uploads and the slow-phase retry worker.

2 One car, start to finish

A simple order on lane 1, with the times you would see on a live feed. The webhook goes out about 3 seconds after the crew's closing line, plus the Gemini call.

0.0 s	event	<code>{"type":"vehicle_arrived"}</code> (if HME sends vehicle events)
1.1 s	Crew	Welcome to Sandbox Burger. What can I get for you today?
5.6 s	Customer	Hi. Can I get a double cheeseburger and a medium fries?
9.6 s	Crew	Sure. A double cheeseburger and a medium fry. Anything else?
14.6 s	Customer	No, that's it.
16.2 s	Crew	Your total is \$8.28. Please pull forward.

WHEN	WHO	WHAT HAPPENS
0.0 s	Tracker	The arrival is noted. A conversation opens on the arrival plus the greeting, or on any customer line.
1.1 s	Tracker	<code>IDLE → ACTIVE</code> . A new <code>order_id</code> is minted right now and kept for this car whatever happens later.
each line	Deepgram	Interim words show on screen; the final utterance (with word confidences) goes to the tracker.
16.2 s	Tracker	"Pull forward" is a crew closing cue: <code>ACTIVE → CLOSING</code> . A 3 s settle timer starts (<code>CLOSE_SETTLE_S</code>).
~20 s	Tracker	No customer speech in 3 s: <code>FINALIZED</code> . The reopen window (20 s) starts.
~20 s	Gemini	One call with the compact menu and the utterances. Returns events: <code>ADD dbl_cheese</code> , <code>ADD fries (size medium)</code> , <code>READBACK</code> . The spoken total (\$8.28) is read from the crew's line by code.
~21 s	Code	<code>replay()</code> builds two lines; post-process puts both in <code>items</code> , computes \$8.28, sees it matches the crew's total, sets status <code>completed</code> (evidence: the "pull forward" cue), review not required.

~21 s	Deliverer	Outbox row <code>ord_..._v1</code> written; signed POST sent; Serv answers 2xx; row marked delivered.
up to 40 s	Tracker	If the customer says "oh, and a water" before the window closes, the conversation reopens and the same <code>order_id</code> is sent again as version 2 (<code>order.updated</code> , <code>correction_reason: reopened_late_addition</code>).

When there is no closing cue

The tracker never guesses that a car left from a pause alone (plan decision D10). Without a closing cue, a conversation finalizes when the next car starts, after 45 s of silence (`IDLE_TIMEOUT_S`), when a `vehicle_departed` event arrives, or at the 6 minute cap. That is why the 95th percentile close time is about 45 s: those orders wait for the idle timer. HME vehicle events would shorten that tail.

When the connection drops

The lane survives reconnects. An open conversation is held for 180 s (`RECONNECT_GRACE_S`). If the base station comes back, the order continues and is flagged `stream_gap` (which also asks for review). If not, it is finalized with `stream_interrupted` . A dropped connection never decides the outcome by itself.

3 Sandbox vs the real setup

Everything after the input layer (tracker, extraction, build, delivery) is what production would run. What differs is what feeds it, where it runs, and the values Serv and HME have not given us yet. Every one of those values lives in `config/sandbox.ts`, shows a yellow Placeholder badge in the UI, and adds the `placeholder_values` flag to orders.

PIECE	SANDBOX TODAY	REAL SETUP
Audio source	Browser simulator, Test Lab, file replays, or a fake base station (<code>pnpm feed replay -via ws</code>)	HME base stations, one stream per lane
Wire format	Placeholder: <code>/hme/v1/stream?lane=&codec=&rate=&channels=</code> , audio as binary frames, events as JSON text	Whatever HME documents. Only <code>src/input/hme/</code> changes
Authentication	Per-store bearer tokens we issue (<code>pnpm pipeline token create</code>)	Confirm with HME; add mTLS, an IP allowlist or signed connects, plus rotation
Hosting	One laptop, <code>127.0.0.1</code> , no TLS	A long-running container (ECS Fargate or similar) behind TLS, sized per number of lanes
Storage	SQLite + local disk under <code>.data/</code>	Postgres for orders and the outbox; S3 for audio and raw capture, same key layout
Webhook target	The app's own mock receiver, <code>/api/mock-webhook</code>	Serv's endpoint (for example API Gateway + Lambda, section 8). Only <code>WEBHOOK_URL</code> and <code>WEBHOOK_SECRET</code> change
Clock	Our receive time, NTP offset checked at start	The same, unless HME sends source timestamps (the parser already keeps them)
Store and lane IDs	<code>store_demo_001</code> , <code>lane_1</code>	Serv's site and lane IDs (store from the token, lane from the connection)
Menu	Invented "Sandbox Burger", 27 items	The brand's real menu, with regional names and promos
Tax	<code>TAX_RATE=0</code>	Per site; otherwise <code>total_mismatch</code> fires on most real orders
Providers	Deepgram free credit; Gemini free tier with a daily cap	Paid tiers (free-tier Gemini prompts may be used by Google, not acceptable for real customer audio)
Monitoring	UI and logs only	Alerts on dead letters, failed sessions and provider errors

Your question: do we connect to HME's WebSocket?

Possibly, but the code assumes the opposite direction, and that is worth confirming with HME first.

WHAT THE CODE ASSUMES NOW

HME connects to us. We run the WebSocket server; each base station dials out to our URL. This matches HME's installation guide, which only asks the installer for a **provider URL and port**: that is the shape of a device that pushes to a provider, not one that waits to be called. It is also easier for stores: no inbound firewall holes at each restaurant.

IF HME EXPOSES A SOCKET INSTEAD

We connect to HME. Then we add a small WebSocket *client* per lane in `src/input/hme/` that dials HME, reconnects with backoff, and feeds the same `HmeConnection` class. Everything from decoding onward is unchanged. Our server-side token check would be replaced by whatever credentials HME gives us.

Either way, the questions for HME are the same:

1. Who opens the connection, and to what URL? One connection per lane, or per store with lanes multiplexed?
2. Audio format: codec, sample rate, mono or a separate crew and customer channel? (Separate channels make speaker roles nearly free and much more accurate.)
3. Does the stream send vehicle-detection events (car at the speaker post, car left)? With what message format and timestamps?
4. Does audio stop when no car is present? (The code handles both: a burst after a pause is placed by its own arrival time.)
5. Authentication: tokens, client certificates, IP ranges?
6. Reconnect behaviour and keepalives. (The simulator assumes HME retries at 2, 4 and 8 s.)

Nothing has been tested against a real base station yet. The raw capture of every message means the first real session can be replayed through the pipeline as often as needed (`pnpm feed replay-raw`) once we have one.

4 What part of the code does what

One repository, two programs that share one engine. `packages/pipeline` holds all the logic and the feed service; `apps/web` is the UI and imports the same package, so the UI can never drift from real behavior.

The live path, in the order a message travels

FILE OR FOLDER	WHAT IT DOES
<code>server/serve.ts</code>	Starts the feed service: checks the NTP offset, opens the database, wires the ingest server to the lane manager, raw capture and the live view.
<code>server/ingest-server.ts</code>	The WebSocket endpoint. Before accepting: per-IP failure limit, token check, lane allowed, format valid. After: message size limits, one connection per store+lane, ping every 20 s, rate limit, revocation, operator stop.
<code>input/auth/</code>	Ingest tokens: create, hash, check, revoke. The <code>IngestAuth</code> interface is where mTLS or an allowlist would plug in.
<code>input/hme/</code>	The only HME-specific code. <code>handshake.ts</code> reads the format from the URL, <code>messages.ts</code> parses JSON events (never throws, keeps unknown messages), <code>connection.ts</code> turns one socket into source messages with timing.

<code>input/decoders.ts</code> , <code>pcm.ts</code> , <code>ogg.ts</code>	Any wire codec (PCM, mu-law, a-law, Opus, or MP3/AAC/WAV/FLAC through ffmpeg) to 16 kHz PCM per channel.
<code>input/file-replay.ts</code> , <code>scenario.ts</code> , <code>ws-replay.ts</code>	Make a file behave like a live feed: pauses, drops, vehicle events, codecs. <code>ws-replay</code> plays a file as a fake base station over the real socket.
<code>data/raw-sink.ts</code> , <code>store/raw-capture.ts</code>	Saves every incoming message before decoding (zstd parts every 60 s or 5 MB).
<code>lane/manager.ts</code>	Routes messages to one <code>LaneSession</code> per store and lane; keeps the lane alive across reconnects.
<code>lane/lane.ts</code>	The heart of the live path: owns the transcriber and tracker for one lane, turns tracker decisions into finalize calls, handles reopen, late evidence, operator stop and the no-speech rules (E1, E2, E9).
<code>lane/deepgram-stream.ts</code>	Deepgram live connection per lane (Nova-3, endpointing 300 ms, utterance end 1 s, menu names as keyterms), closed when idle for 30 s and reopened on resume.
<code>lane/file-transcriber.ts</code> , <code>timed-stream.ts</code> , <code>script-transcriber.ts</code>	For files: transcribe once with Deepgram's prerecorded API (cached) and release each utterance as its audio "arrives". The script transcriber uses the fixture's exact words, free.
<code>lane/tracker.ts</code>	The conversation state machine and all its timers. Pure: given utterances, events and clock ticks, returns decisions with their reasons.
<code>lane/online-roles.ts</code> , <code>transcribe/roles.ts</code>	Who is crew and who is the customer, from channels, diarization or wording.
<code>segment/cues.ts</code> , <code>segment.ts</code>	The phrase lists (greetings, closings, "they left", "that's it") and gap scoring the tracker uses.
<code>orders/finalize.ts</code>	One finished conversation to order versions: extract, <code>replay()</code> , post-process, format the payload, store it, enqueue it.
<code>extract/gemini.ts</code> , <code>prompt.ts</code> , <code>llm-schema.ts</code> , <code>validate.ts</code>	The Gemini call (schema-constrained JSON, one repair retry, daily cap, cache), and validation of every catalog ID. <code>fuzzy-extractor.ts</code> is the no-LLM fallback; <code>oracle.ts</code> replays hand-written events for the eval.
<code>build/replay.ts</code>	Pure <code>replay(events)</code> : applies adds, replacements, quantity changes and cancels in order. Also runs in the browser.
<code>postprocess/postprocess.ts</code> , <code>outcome.ts</code>	Buckets, combos, totals, flags; the outcome from evidence and the review reasons. Pure.
<code>webhook/payload.ts</code> , <code>corrections.ts</code>	Formats schema 2.0, and builds version N+1 from version N.
<code>webhook/deliver.ts</code> , <code>signing.ts</code>	The outbox, retries and ordering between versions; Standard Webhooks signing and verification.

<code>webhook/mock-receiver.ts</code>	The sandbox's stand-in for Serv: verifies, dedupes, keeps the highest version, can fail on purpose. A good model for the real receiver.
<code>store/db.ts</code>	SQLite schema: runs, orders and versions, outbox, attempts, mock inbox, live events, Test Lab results.

Around the live path

FILE OR FOLDER	WHAT IT DOES
<code>config/sandbox.ts</code>	Every Serv-dependent setting with its placeholder status.
<code>menu/menu.json</code> , <code>menu/ code</code>	The catalog, aliases and fuzzy matching.
<code>eval/</code>	<code>pnpm eval</code> : Layer A (what was said), Layer B (what was rung up, against POS tickets), live-path checks, the 41-row checklist, the webhook self-check.
<code>fixtures/</code> , <code>sim/</code>	24 scripted conversations rendered to audio with noise, plus the simulator's back end (save a session as a fixture).
<code>data/</code>	The blob store and artifact catalog behind <code>pnpm data find / usage / prune</code> .
<code>apps/web/app/</code>	Pages (<code>/</code> , <code>/runs/[id]</code> , <code>/orders</code> , <code>/testlab</code> , <code>/simulator</code> , <code>/eval</code> , <code>/mock-webhook</code>) and API routes.
<code>apps/web/lib/jobs.ts</code> , <code>instrumentation.ts</code>	The in-process run queue for uploads, and the slow-phase webhook retry worker.
<code>apps/web/components/testlab/</code> , <code>testlab/scenarios/</code>	Test Lab: guided scripts, the scorecard, history.
<code>apps/web/e2e/</code>	Playwright tests, including the monkey test (random clicking for minutes, no errors allowed).

5 The webhook, exactly

One HTTP POST per order version, JSON body, schema 2.0. The body is built once and never changes between retries.

Headers

HEADER	EXAMPLE	MEANING
content-type	application/json	
user-agent	serv-audio-orders/0.1	
webhook-id	ord_01M47MMRZ1J9P2ACY80ARBSQ2A_v1	{order_id}_v{order_version} . Same on every retry; different for each version. The dedupe key.
webhook-timestamp	1791257896	Unix seconds when this attempt was signed. Reject if more than 5 minutes off (replay protection).
webhook-signature	v1,RTg5Y6JZ...DS4=	v1, + base64 HMAC-SHA256 of {id}.{timestamp}.{body} with the shared secret (whsec_...). Space-separated list during secret rotation.
x-delivery-attempt	1	Attempt number for this version, for logging. Not signed.

Body: every top-level field

FIELD	TYPE AND MEANING
schema_version	"2.0"
event_type	order.finalized for version 1, order.updated for every later version
order_id	ord_ + ULID. Minted when the conversation opens; stable across versions
order_version	1, 2, 3... Keep the highest per order
supersedes_version , correction_reason	null on v1. Later: the previous version, and reopened_late_addition , late_evidence or human_review
group_id	Shared by the orders of one car that split the bill ("can we pay separately?"); otherwise null
store_id , lane_id , session_id	Where it came from; session_id is one WebSocket connection
status	completed , cancelled , abandoned or undetermined (section 6)

outcome_evidence[]	What decided the status: type (spoken_cue , vehicle_event , stream_event , silence , order_state , human_review), kind , cue , event , utterance_id , at ; context_only: true when it was only context
review	{required, reasons[]}. Reasons: unclear_items , readback_mismatch , total_mismatch , missing_required_slot , low_audio_quality , stream_gap , transcript_gap , roles_guessed_low_agreement , safety_cap . (outcome_undetermined is still valid in the schema but no longer sent, E11.)
times	started_at , ended_at (when it was spoken), time_basis (receive_clock live, recording_metadata for files, source_clock if HME sends one), received_at , finalized_at (when we processed it)
audio_ref	session_id , sample_start , sample_end (16 kHz samples), archive_uri of the saved FLAC for this version
source	type (hme_ws , file_replay , rtsp), codec_in , channels , channel_roles
items[]	Confidently ordered lines: line_id , catalog_id , name , quantity , size , components[] (combo slots), modifiers[] ({id, action}), unit_price , recognition_confidence , commitment_confidence , source_utterance_ids
needs_review[]	Ordered but unclear what: catalog_id (may be null), raw_text , quantity , size , top 3 candidates , confidences, source lines
not_ordered[]	Mentioned but not ordered, with reason : cancelled , replaced (and replaced_by), declined_upsell , out_of_stock , inquired , uncommitted
combo_opportunities[] , customer_declined_combo	Separate lines that would be cheaper as a meal, with savings . Never converted automatically
flags[]	Facts about the conversation, for example split_order , non_english , truncated_end , stream_gap , no_speech , ended_by_operator , placeholder_values (19 in all)
totals	computed (from catalog prices), spoken_by_crew (or null), currency
overall_confidence	0 to 1
transcript[]	This conversation's lines: id , speaker , speaker_guessed , start_utc , end_utc , text , confidence , non_customer (crew chatter)
processing	stt , extractor , menu_version , pipeline_version , latency_ms

A real payload, trimmed

From `examples/01_simple/webhook-payload.json` . The transcript is cut to two lines; the full file has all six.

```
{
  "schema_version": "2.0",
  "event_type": "order.finalized",
  "order_id": "ord_01M47MMRZ1J9P2ACY80ARBSQ2A",
  "order_version": 1, "supersedes_version": null, "correction_reason": null,
  "group_id": null,
  "store_id": "store_demo_001", "lane_id": "lane_1",
  "session_id": "ses_01M47MMRVYRGMSY1YBNJS2VYN",
```

```

"status": "completed",
"outcome_evidence": [{ "type": "spoken_cue", "kind": "closing", "cue": "pull forward",
                        "utterance_id": "u6", "at": "2026-10-03T18:40:19.435Z" }],
"review": { "required": false, "reasons": [] },
"times": { "started_at": "2026-10-03T18:40:01.120Z", "ended_at": "2026-10-03T18:40:20.555Z",
            "time_basis": "recording_metadata",
            "received_at": "2026-10-06T03:38:15.811Z", "finalized_at": "2026-10-06T03:38:15.912Z" },
"audio_ref": { "session_id": "ses_01M4...", "sample_start": 17920, "sample_end": 328880, "archive_uri": null },
"source": { "type": "file_replay", "codec_in": "pcm_s16le", "channels": 1, "channel_roles": ["mixed"] },
"items": [
  { "line_id": "e1", "catalog_id": "dbl_cheese", "name": "Double Cheeseburger", "quantity": 1,
    "size": null, "modifiers": [], "unit_price": 5.49,
    "recognition_confidence": 0.99, "commitment_confidence": 1, "source_utterance_ids": ["u2"] },
  { "line_id": "e2", "catalog_id": "fries", "name": "Fries", "quantity": 1,
    "size": "medium", "modifiers": [], "unit_price": 2.79,
    "recognition_confidence": 0.99, "commitment_confidence": 1, "source_utterance_ids": ["u2"] }
],
"needs_review": [], "not_ordered": [], "combo_opportunities": [], "customer_declined_combo": false,
"flags": ["placeholder_values"],
"totals": { "computed": 8.28, "spoken_by_crew": 8.28, "currency": "USD" },
"overall_confidence": 0.99,
"transcript": [
  { "id": "u1", "speaker": "crew", "speaker_guessed": true, "start_utc": "2026-10-03T18:40:01.120Z",
    "end_utc": "2026-10-03T18:40:04.880Z",
    "text": "Welcome to Sandbox Burger. What can I get for you today?", "confidence": 0.998 },
  { "id": "u2", "speaker": "customer", "speaker_guessed": true, "start_utc": "2026-10-03T18:40:05.600Z",
    "end_utc": "2026-10-03T18:40:09.200Z",
    "text": "Hi. Can I get a double cheeseburger and a medium fries?", "confidence": 0.989 }
],
"processing": { "stt": "deepgram/nova-3", "extractor": "gemini/gemini-3.5-flash-lite",
                "menu_version": "sandbox-1", "pipeline_version": "0.1.0", "latency_ms": 5 }
}

```

A version 2 carries the whole order again, not a diff: same `order_id`, `order_version: 2`, `event_type: "order.updated"`, `supersedes_version: 1`, and a `correction_reason`. The receiver can simply replace what it holds.

Size: the transcript is the bulk of the body (a few KB for a normal order). If Serv does not want transcripts for size or privacy reasons, it is one field to drop.

6 Outcomes, including abandoned

Yes, abandoned orders are sent, like every other outcome. The status answers one question, "how did this conversation end?", and is set only from evidence. Whether a person should check the order is a separate question, answered by `review`, which can be true on any status.

STATUS	SET WHEN	EXAMPLE EVIDENCE
completed	A crew closing cue ("pull forward", "see you at the window") or the customer's "that's it" acknowledged by the crew, after at least one item	spoken_cue / closing / "pull forward"
cancelled	The customer cancelled the whole order	spoken_cue / cancel, or order_state / all_items_removed

abandoned	The car left before a completion: a <code>vehicle_event</code> / <code>vehicle_departed</code> <code>vehicle_departed</code> event, the crew saying so ("they drove off"), or the next car's <code>vehicle_arrived</code>	
undetermined	No evidence either way (for example, the audio stops with no cue and no vehicle event)	Only <code>context_only</code> entries

WHY PAUSES DO NOT DECIDE

A pause, silence or a dropped connection is recorded as `context_only` evidence and never sets completed or abandoned by itself (plan decision D10). A crew member can step away mid-order; a connection can drop while the car is still there. Guessing would send confident but wrong outcomes.

Exactly what is sent in each abandoned-style case

WHAT HAPPENED IN THE LANE	SENT?	AS
Customer ordered some items, then the car left (departure event or the crew says so)	Yes	<code>abandoned</code> , with whatever items were heard
Car arrived and left, nobody spoke at all (E1)	Yes	<code>abandoned</code> , no items, flag <code>no_speech</code> , no review, no Gemini call
Customer spoke, then the next car arrived (vehicle event)	Yes	<code>abandoned</code> , with the next car's arrival as evidence
Customer spoke, then audio stopped with no cue and no vehicle event	Yes	<code>undetermined</code> (E11: not flagged for review on that alone)
Car arrived, nobody spoke, then a pause, a lost connection or End session (E9)	No	Without a departure the status would be <code>undetermined</code> with nothing in it
No vehicle event and no speech (E2)	No	Nothing opened
Car arrived, the crew greeted it, the customer never spoke, the car left	No	Gap, see below

GAP FOUND WHILE WRITING THIS: A GREETED CAR THAT DRIVES OFF SILENTLY IS NOT SENT

Checked on October 6 with a throwaway lane test: `vehicle_arrived` , the crew's "Welcome, what can I get for you?", `vehicle_departed` gives **no order**. A car that left without anyone speaking is sent (E1). The cause is in `lane/lane.ts` (`runFinalize`): a conversation with no customer speech is dropped, and the E1 exception only applies when there was no speech at all. From Serv's point of view the greeted car is the more interesting abandoned visit (someone was at the post and left), so it should be sent like E1: `abandoned` , no items, `no_speech` (or a new `no_customer_speech` flag), no review, no Gemini call.

This is a small change plus a lane test, and it needs your decision (call it E13) since it changes what Serv receives. Without vehicle events, nothing changes: a greeting with no customer and no departure still sends nothing.

Without vehicle events

If HME does not send vehicle events, abandonment can only come from what is said ("they left", or the crew greeting a new car mid-order together with an arrival). Otherwise the visit ends `undetermined` : still sent, with items and times, so Serv can match it to the POS. Vehicle events are the single biggest improvement for outcomes, which is why they are on the list of questions for HME.

7 How duplicates are prevented

Webhooks are **at least once** by nature: if Serv processes an order but its response is lost (a timeout, a dropped connection), we cannot tell the difference from a failure and must retry. So the guarantee is built in two halves: we never create a second identity for the same order version, and the receiver ignores a version it has already seen.

Our side (the sender)

MECHANISM	WHAT IT PREVENTS
One <code>order_id</code> per car, minted when the conversation opens	A reopen, late evidence or a human correction becomes version 2 of the same order , never a new order.
<code>webhook-id = {order_id}_v{version}</code>	Retries of one version carry the same ID; each version has its own. The receiver needs no other key.
Outbox primary key on <code>webhook_id</code> , insert-or-ignore	Enqueueing the same version twice (a restart, a double finalize) is a no-op.
Orders table primary key (<code>order_id</code> , <code>version</code>)	A version is stored once.
Atomic claim (<code>status pending</code> → <code>delivering</code>)	The inline sender and the retry worker can never send the same row at the same moment.
Frozen body	Every retry sends byte-identical JSON; only the timestamp and signature headers are new.
Version ordering	Version N waits until N-1 is delivered, failed or dead, so <code>order.updated</code> never arrives before the <code>order.finalized</code> it corrects.
Stuck-row recovery	When the worker starts after a crash, rows left <code>delivering</code> for over 2 minutes go back to pending. They may be sent again, which is exactly what receiver dedupe absorbs.
Eval check	<code>pnpm eval</code> counts duplicate versions and premature reopens on every fixture; checklist rows 27 and 28 run a signed delivery and a duplicate against the mock receiver.

Serv's side (the receiver): two rules

1. DEDUPE ON WEBHOOK-ID

If you have already accepted this ID, answer **200** and do nothing. Never answer 4xx or 5xx to a duplicate, or we will keep retrying.

2. KEEP THE HIGHEST VERSION

Store per `order_id`. Accept a version only if it is higher than the one held; an older or equal one is a duplicate. One conditional write does both rules (section 8).

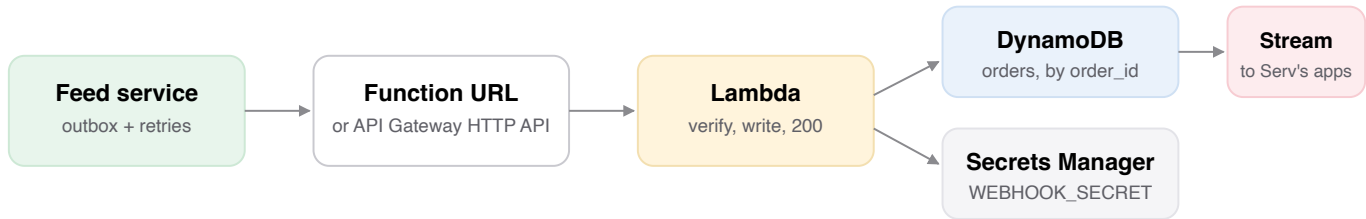
The sandbox's mock receiver (`webhook/mock-receiver.ts`) does exactly this, and you can watch it on `/mock-webhook` : send an order, use Resend, and the second arrival shows as *duplicate*.

What can still produce a second order for the same car

CASE	WHAT TO DO
The tracker splits one car into two conversations (a long pause mid-order read as a new car)	A segmentation error, not a delivery bug: two <code>order_id</code> s. Measured by the eval (74 of 74 fixture conversations found, none extra). Real audio may differ.
Someone replays a captured session or re-runs a file into the same webhook	Mints new <code>order_id</code> s by design (it is a new run). Point replays at the mock, not at Serv. If needed, Serv can add a soft check on <code>store_id + lane_id + times.started_at</code> .
A split bill (<code>group_id</code> set)	Not a duplicate: two real orders from one car, linked by <code>group_id</code> .
The feed service restarts mid-conversation	The open conversation is lost, not duplicated (lane state is in memory). Already-built orders are safe in the outbox. In production, run one owner per lane and drain on deploy.

8 Receiving the webhook on AWS Lambda

Nothing changes on our side except two settings: `WEBHOOK_URL` (the Lambda's URL) and `WEBHOOK_SECRET` (a `whsec_...` secret both sides share). The work is on the receiving side.



Steps

1. **Generate a secret** in the `whsec_<base64>` form (our `generateSecret()` makes one) and store it in Secrets Manager. Give the same value to the feed service as `WEBHOOK_SECRET`.
2. **Create a DynamoDB table** `orders` with partition key `order_id` (string). On-demand billing is fine.
3. **Create the Lambda** (Node.js 22, 256 MB, 10 s timeout) with the handler below and the `standardwebhooks` npm package. Give its role `dynamodb:PutItem` on the table and read access to the secret.
4. **Expose it** with a Lambda Function URL (auth type NONE: the signature is the authentication) or an API Gateway HTTP API route `POST /serv/orders`, if Serv wants WAF, throttling or a custom domain.
5. **Point the feed service at it:** `WEBHOOK_URL=https://...`. Check with `pnpm pipeline settings`, then send a fixture: `pnpm pipeline run fixtures/audio/...` and watch the deliveries panel on the run page.
6. **Test failure paths:** Resend an order (expect 200 and no new row), and make the Lambda return 500 once (we retry with backoff, then the order lands).

The handler

```
// index.mjs (Node.js 22, ESM). npm i standardwebhooks @aws-sdk/client-dynamodb
import { Webhook } from "standardwebhooks";
import { DynamoDBClient, PutItemCommand, ConditionalCheckFailedException } from "@aws-sdk/client-dynamodb";

const db = new DynamoDBClient({});
const wh = new Webhook(process.env.WEBHOOK_SECRET); // whsec_... (load from Secrets Manager in prod)

export const handler = async (event) => {
  // Verify against the exact bytes we sent; never re-serialize the JSON first.
  const body = event.isBase64Encoded ? Buffer.from(event.body, "base64").toString("utf8") : event.body;
  let order;
  try {
    order = wh.verify(body, event.headers); // checks signature + 5 min timestamp window
  } catch {
    return { statusCode: 401, body: "bad signature" }; // 4xx: we will not retry
  }

  try {
    // One conditional write = dedupe on webhook-id AND keep the highest version.
  }
}
```

```

await db.send(new PutItemCommand({
  TableName: "orders",
  Item: {
    order_id:      { S: order.order_id },
    order_version: { N: String(order.order_version) },
    webhook_id:    { S: event.headers["webhook-id"] },
    status:        { S: order.status },
    store_id:      { S: order.store_id },
    lane_id:       { S: order.lane_id },
    payload:       { S: body },
  },
  ConditionExpression: "attribute_not_exists(order_id) OR order_version < :v",
  ExpressionAttributeValues: { ":v": { N: String(order.order_version) } },
}));
} catch (e) {
  if (e instanceof ConditionalCheckFailedException) {
    return { statusCode: 200, body: '{"duplicate":true}' }; // same or older version: already have it
  }
  throw e; // an unhandled error gives a 5xx: we retry
}
return { statusCode: 200, body: '{"received":true}' };
};

```

Rules that matter

- **Answer fast.** We time out after 10 s and retry. Store the payload and return 200; do POS matching or anything slow downstream (DynamoDB Streams, or SQS FIFO with `MessageDeduplicationId = webhook-id`).
- **Status codes steer our retries.** Any 2xx is success. 408, 429 and 5xx (and timeouts) are retried, honoring `Retry-After`: about 1, 2, 4, 8, 16, 32 s, then 5 min, 30 min, 2 h, 5 h, 10 h, 10 h, then dead letter. Any other 4xx is treated as permanent: return 401 only for a bad signature, and 400 only for a body you can never accept.
- **Verify the raw body.** API Gateway and Function URLs pass the body through as a string; parsing and re-stringifying it before verifying will break the signature.
- **Lowercase headers.** Both Function URLs and HTTP APIs deliver them lowercased, which is what the library expects.
- **Secret rotation.** The signature header may hold two signatures (space-separated) while a new secret rolls out; the library accepts either.
- **Could the feed service itself run on Lambda?** No: it holds long-lived WebSockets and per-lane state for hours. It belongs on a container (ECS Fargate) with the outbox in Postgres. Only the receiver fits Lambda well.

9 Testing it yourself

A local copy is running for you on the `v2.1` branch:

Web app	<code>http://localhost:3000</code> (started with <code>ENABLE_DEV_ROUTES=true pnpm dev</code>)
Feed service	<code>127.0.0.1:8787</code> (<code>ENABLE_DEV_ROUTES=true pnpm feed serve</code>)
Test Lab	<code>http://localhost:3000/testlab</code>
Mock webhook	<code>http://localhost:3000/mock-webhook</code> : every payload received, signature check, duplicates, failure toggles

COST

Microphone tests stream to Deepgram live and spend real Deepgram credit, billed per minute. Text mode ("type instead of speaking", under Advanced) is free. Gemini stays on the free tier with its daily cap.

Suggested order

TEST	WHAT IT PROVES
Test Lab scenario 1, typed	The tracker, extraction and webhook end to end, free
Scenarios 1, 4 and 6 with your voice	Deepgram live, speaker roles, a late add-on (version 2), a drive-off (abandoned). This is the remaining hand test for the v2.1 release.
Free play	Anything you like; afterwards you pick what was really ordered and get a scorecard
<code>/mock-webhook</code> , set "fail 500" for 2 requests, then run a test	Retries: the deliveries panel on the run shows each attempt, then success
Resend an order from its run page	Dedupe: the mock shows the second arrival as duplicate, and keeps one order
<code>/orders</code> , filter Review: Flagged	Which orders the system would ask a person to check

The scorecard says where each mistake came from: **heard wrong** (transcription), **understood wrong** (extraction), or **timing** (the tracker split, merged or failed to reopen a conversation). It finds out by replaying your session with the script's exact words through the same tracker and extractor.

10 Gaps and next steps

PRIORITY	GAP	NEXT STEP
High	HME interface unknown (direction, format, events, auth)	The questions in section 3. Then replace <code>src/input/hme/</code> and replay the first real session from raw capture.
High	Greeted car that drives off silently is not sent	Decide E13; then a small change in <code>lane.ts</code> and a lane test (section 6).
High	Only synthetic audio so far; thresholds uncalibrated	Record the held-out set (8 to 12 real voices), then real HME audio with POS tickets.
Medium	Serv has not reviewed the schema	Send them section 5 and an example; ask about the transcript field and tax.
Medium	Sandbox infrastructure (SQLite, one machine, no auth, no alerts)	Container + Postgres + S3; alerts on dead letters; sign-in on the web app.
Medium	Close latency tail of about 45 s without a closing cue	Vehicle events from HME, or tune <code>IDLE_TIMEOUT_S</code> on real traffic.
Low	Live speech barely exercised; stereo never sent to Deepgram	Test Lab with voices; one <code>pnpm eval --layout stereo run</code> .
Low	Live and Review pages disabled (E12)	Code kept; re-enable when there is real traffic to watch and review.

A Commands and glossary

Commands

```
pnpm dev                # web app + mock webhook, localhost:3000
pnpm feed serve          # live service, 127.0.0.1:8787
pnpm pipeline token create --store s1 --lanes lane_1, lane_2    # ingest token for a base station
pnpm feed replay lane_stream_a --speed 1                        # a recording as a live feed
pnpm feed replay 01_simple --via ws                             # ...as a fake base station over the real socket
pnpm feed sim-check      # the five acted scenarios, typed, over the real endpoint
pnpm pipeline run path/to/file.mp3                             # one file through the lane, delivered to WEBHOOK_URL
pnpm pipeline resend <order_id>                                # resend a failed or dead-lettered delivery
pnpm pipeline settings   # which values are still placeholders
pnpm eval                # accuracy on every fixture (asks before spending Deepgram)
pnpm data find --order <order_id>                              # everything kept for one order
pnpm test && pnpm test:e2e                                       # unit and route tests; Playwright
```

Glossary

Base station	HME's box at the restaurant that carries the headset audio. It would stream to us.
Lane	One drive-thru lane; one audio stream, one tracker, one order at a time.
Session	One WebSocket connection. A lane outlives its sessions (reconnects).
Conversation	One car's exchange at the speaker post; becomes one order (or several, for a split bill).
Tracker	The per-lane state machine that decides when a conversation starts and ends.
Event	What the LLM returns: <code>ADD</code> , <code>REPLACE</code> , <code>CANCEL_ORDER</code> and 12 more, with catalog IDs. Code turns events into the order.
Version	Each send of one order. v1 is <code>order.finalized</code> ; corrections are <code>order.updated</code> .
Outbox	The table every webhook is written to before sending, so nothing is lost on a crash.
Dead letter	A delivery that failed every retry (about a day). Resend from the UI or CLI.
Review	The flag that says a person should check this order, with reasons. Independent of status.
Placeholder	A value we invented until Serv or HME provides the real one; flagged on every order.
Fixture	A scripted test conversation with a known right answer.
E1 to E12	The v2.1 decisions, listed in the README.